SWINBURNE UNIVERSITY OF TECHNOLOGY

COS30082
Applied Machine Learning

Lecture 11
Large Language Models (LLMs)

Topics

- What are Large Language Models?
- The Transformer Architecture
- Examples of LLMs: BERT and GPT
- How are LLMs Trained?
- Limitations and Challenges of LLMs

What Are Large Language Models (LLMs)

- Large Language Models (LLMs) are advanced AI models, created to comprehend and produce human language, code, and beyond.
- Large = billions of parameters, trained on hundreds of billions of words.
- Built on the Transformer architecture (2017).
- Versatile in tasks, they excel in accuracy, fluency, and style.
- Examples: GPT-4, Claude, Gemini, LLaMA, BERT.
- A single model learns language patterns so well that it can be reused for many tasks.

What Are Large Language Models (LLMs)

LLMs are Deep Neural Networks (DNNs) for language and must satisfy all of the following:

- Deep Transformer-based architecture (not RNN, LSTM…)
- Very large scale (hundreds of millions to billions of parameters)
- Pre-trained on massive corpora (books, web, code, etc.)
- General-purpose (not task-specific)

What Can LLMs Do?

- Generation — write essays, emails, stories, reports, and code.
- Understanding — classify text, analyse sentiment, and recognise named entities.
- Transformation — translate, summarise, rewrite, and change writing style.
- Reasoning — support problem solving, maths, planning, and decision-making.
- Conversation — power chatbots, tutoring systems, and customer support.
- Tool use and agents — call APIs, browse the web, write and run code, and complete multi-step tasks.

Current Popular LLMs

SWINBURNE UNIVERSITY OF TECHNOLOGY SARAWAK CAMPUS

Commercial (API + chat)

- Anthropic's Claude: Reasoning, coding, long context, image understanding
- OpenAI's GPT: General-purpose use, multimodal input, image generation
- Google's Gemini: Long context, multimodal input, image generation, Google integration

Open-weight (free to download & run)

- Meta's LLaMA: most widely used open base model
- DeepSeek: very strong reasoning, open weights
- Alibaba's Qwen: multilingual, strong on Chinese + code (Qwen is shortened from Qianwen, meaning "a thousand questions.")

Topics

- What are Large Language Models?
- The Transformer Architecture
- Examples of LLMs: BERT and GPT
- How are LLMs Trained?
- Limitations and Challenges of LLMs

Before Transformers: RNNs & LSTMs

Older models such as RNNs and LSTMs process text one token at a time, from left to right.

- each step depends on the previous step, so training is difficult to parallelise.
- information from earlier words or earlier sentences may become weaker by the time the model reaches the later text.

  Example: Mary picked up the red key. She used it to open the door.

  The model needs to know it refers to the red key from the previous sentence.

Need: an architecture that can look at all tokens in the given context together and learn relationships between distant words more effectively.

Transformer Architecture

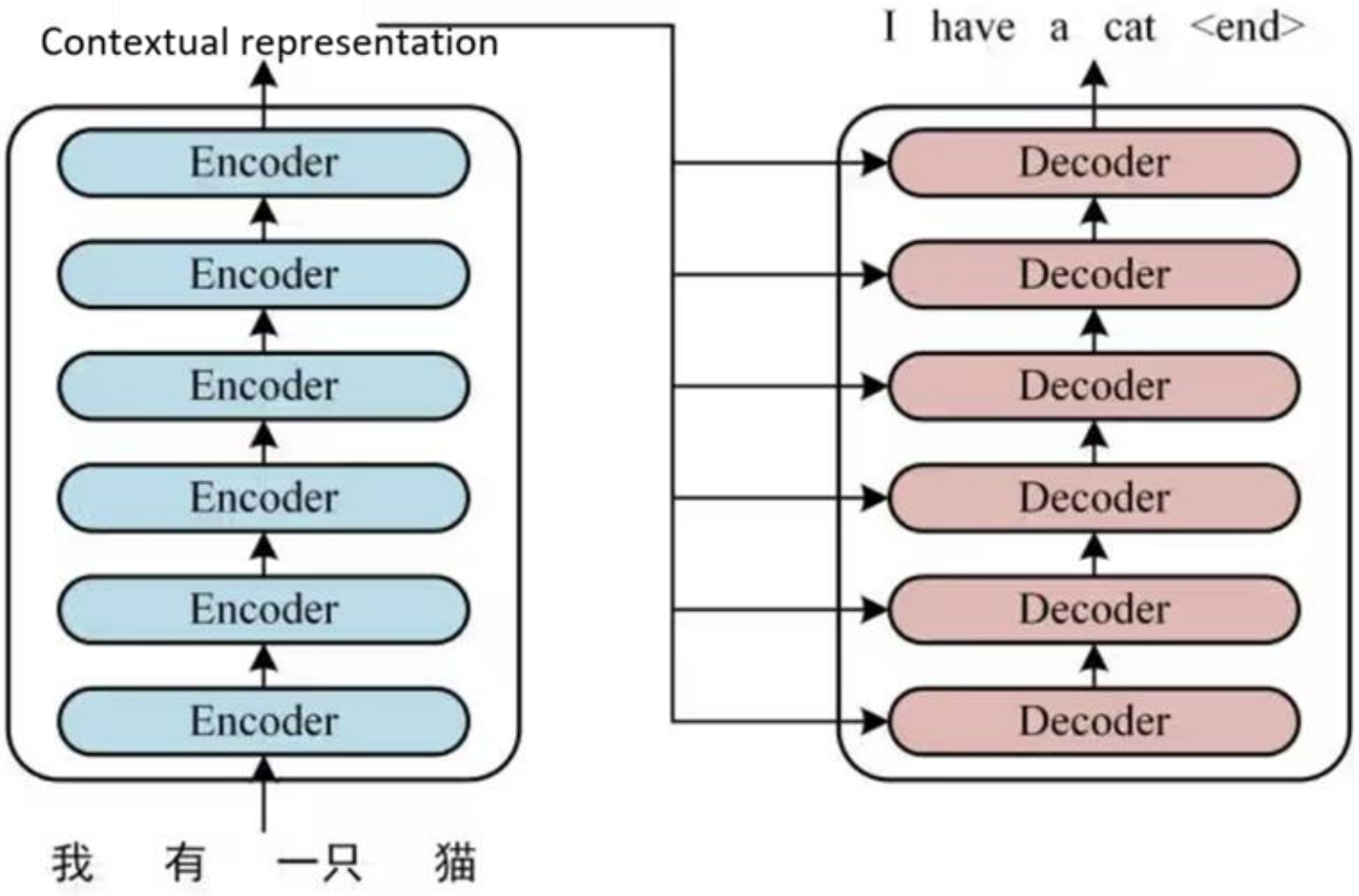
- A neural network architecture used in modern LLMs.
- Uses self-attention to connect related words/tokens.
- Can process text in parallel.
- Better at handling long-range relationships in text.

The Transformer architecture was introduced in the 2017 paper:

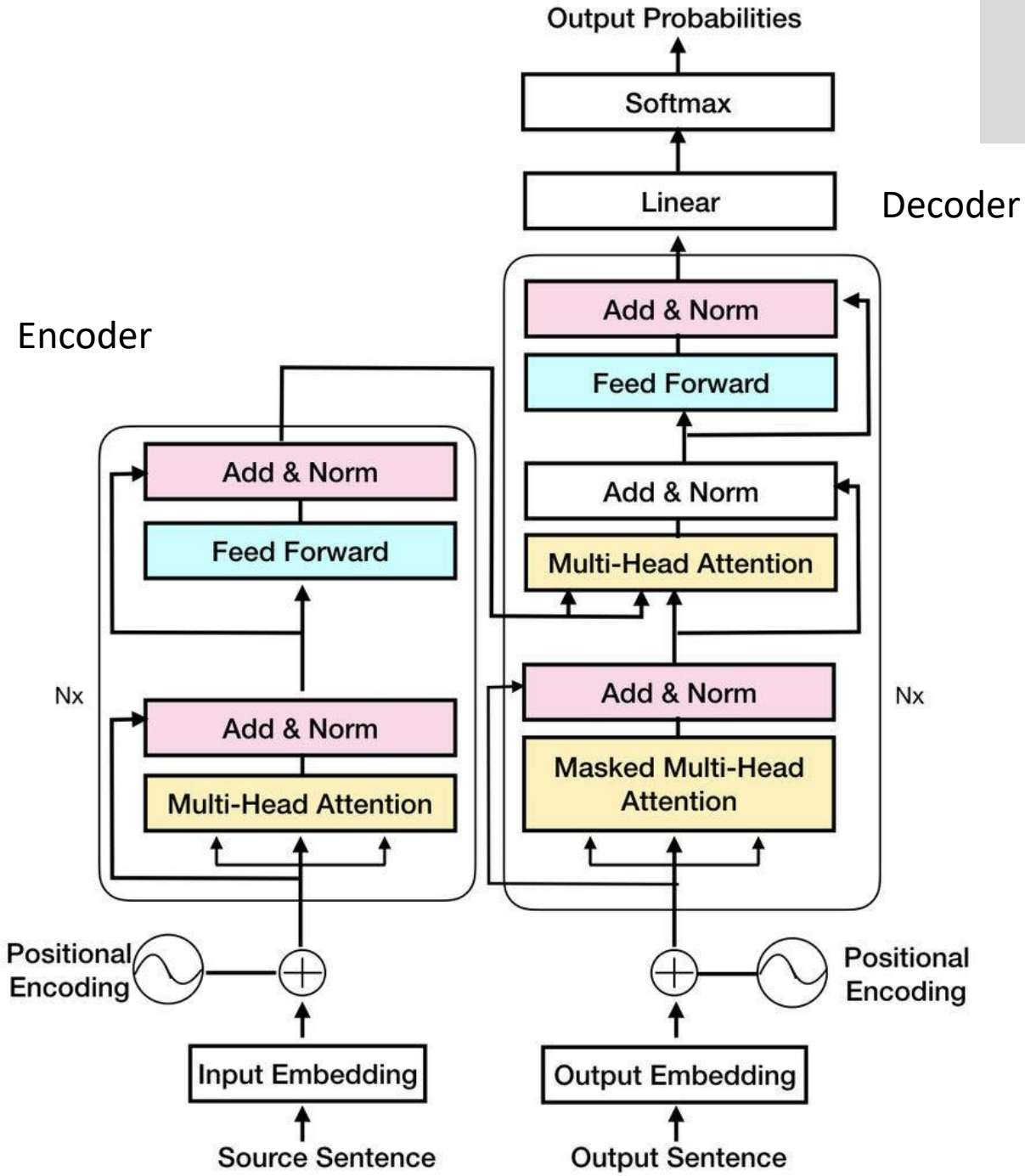
> Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

Transformer Architecture

There are 6 layers encoder followed by 6 layers decoder.



Paper: Attention is all you need



Encoder

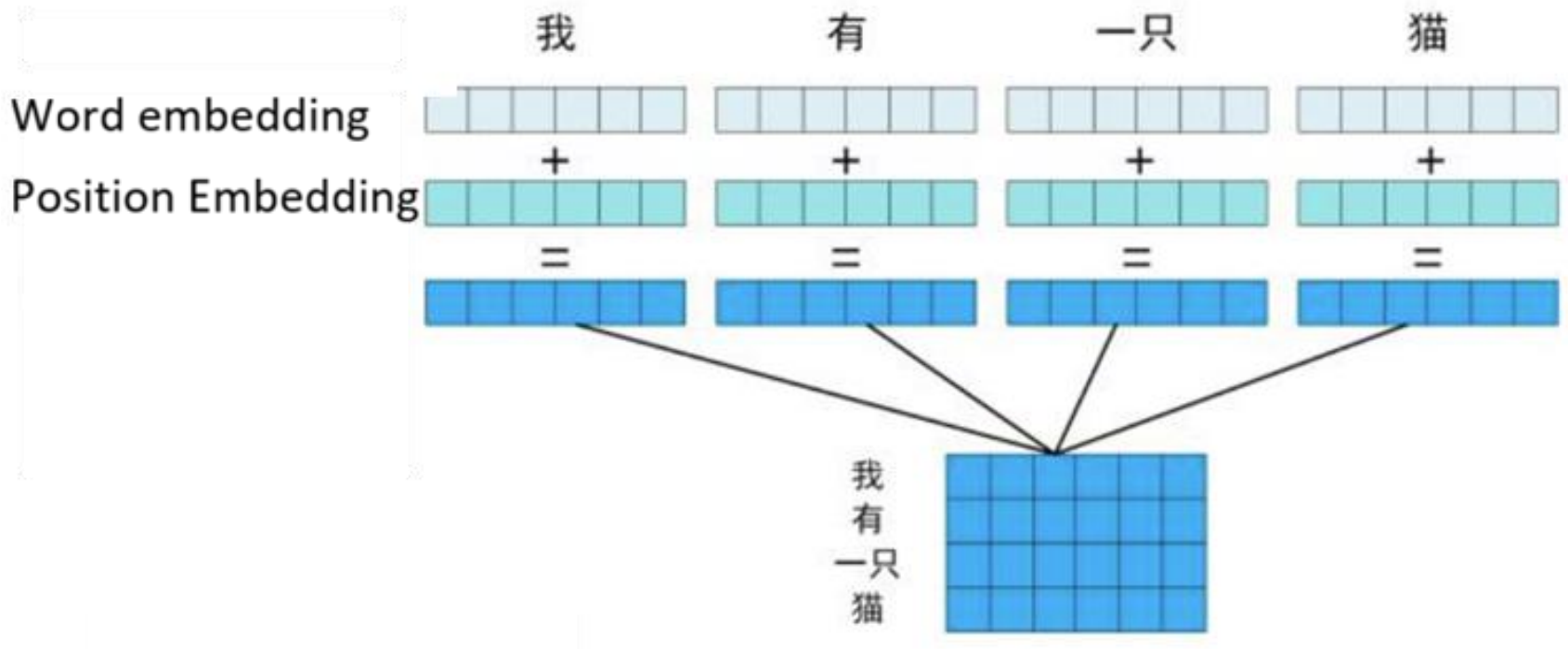
1.Input embeddings + Positional Encoding
→ Turns words into vectors and adds info about word order.

2.Multi-Head Self-Attention
→ Each word **looks at other words** in the input to understand the context.

3.Feedforward Network
→ A small neural network that transforms each word's representation.

4.Add & Layer Norm
→ The input vector is added to the output of the small neural network, and the result is then normalized. Helps with stability and learning.

Input and Position



The Transformer uses mathematical positional vectors, not simple numbers like 1, 2, 3, so the model can understand word order and distance between words.

Self-Attention: Query, Key, Value

- Self-attention uses three learned matrices: Query (Q), Key (K), and Value (V). They are a bit like CNN filters because they learn useful patterns.
- The input vectors are processed using the learned Q, K, and V matrices to produce contextual output vectors.
- Multi-head attention means the Transformer uses multiple sets of Q, K, and V matrices in parallel, so different heads can learn different word relationships.
- The encoder output is a contextual representation that includes word meaning, word order, and relationships with other words.

Attention — Worked Example

Sentence: “The cat sat on the mat because it was tired.”

Each word has its own Q, K, and V vectors.

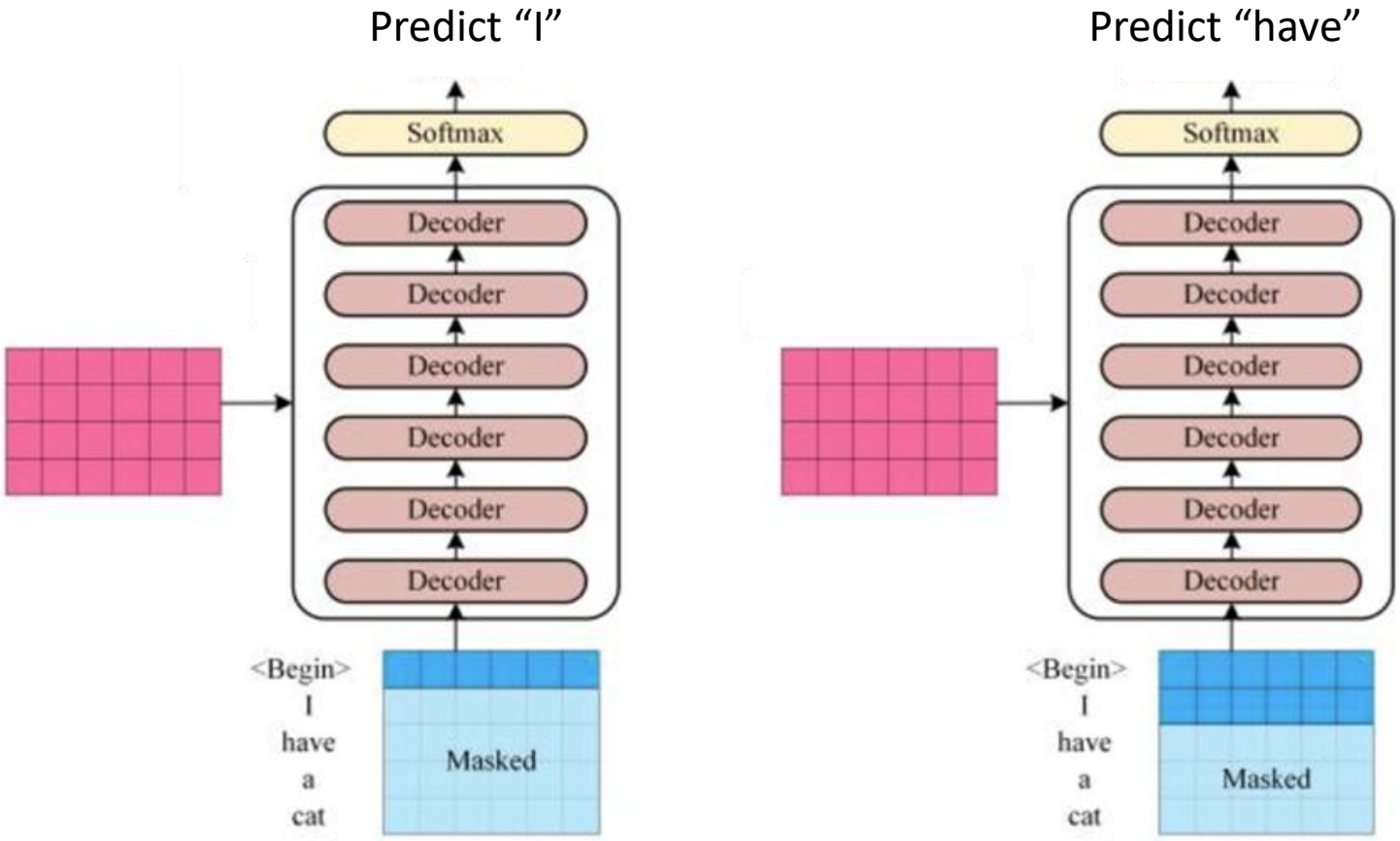
- Query (Q) = what this word is looking for.
- Key (K) = what this word can match with.
- Value (V) = the information this word can pass on.

What does “it” refer to?

- The word “it” sends out a Query: “I’m a pronoun — which noun am I?”
- “cat” advertises a Key that strongly matches → high attention weight.
- “mat” has a weaker matching Key → low weight.
- “it” pulls mostly cat’s Value into its own vector.

- Result: the representation of “it” now contains the meaning of “cat”.
- This happens in parallel for every word in the sentence.

Decoder



Decoder

Language translation from Chinese to English
Chinese: 我有一只猫。
English: <START> → I have a cat. <END>

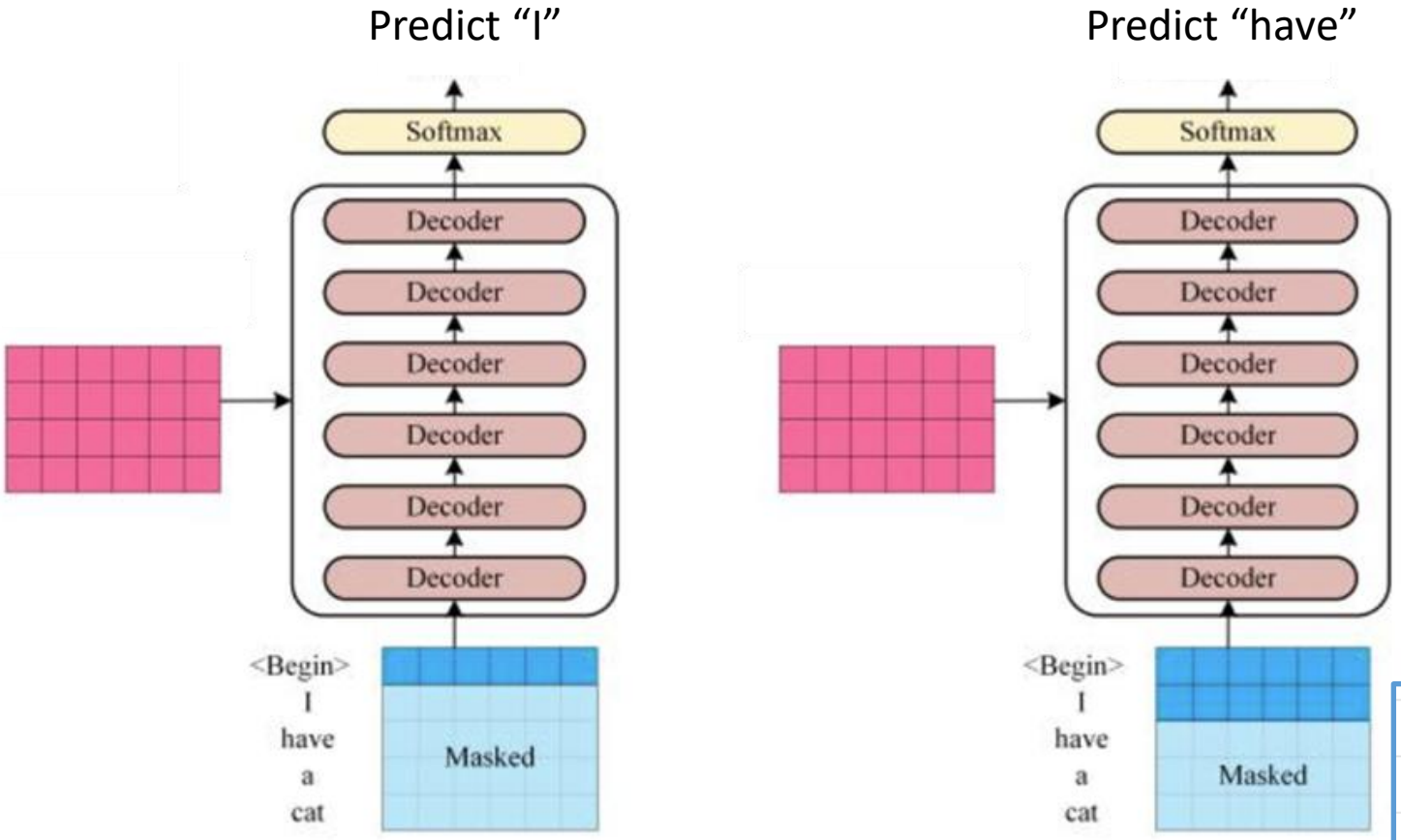
1. The decoder starts with the special token <Begin>. Using <Begin> and the encoded representation, the decoder predicts the first English word: I.

2. Now the decoder uses the previous output “I”. It also continues to use the encoded representation from the encoder. The decoder predicts the next word: "have“.

And continues until it predicts <end>.

Training loss: the predicted output is compared with the correct target word, and the model learns by reducing the difference.

Mask



During training, the whole target sentence is input to the decoder at once, but a mask hides future words. This lets the model predict all next words in parallel, while still only using previous words for each prediction.

<start>	I	have	a	cat.
<start>	I	have	a	cat.
<start>	I	have	a	cat.
<start>	I	have	a	cat.
<start>	I	have	a	cat.

Topics

- What are Large Language Models?
- The Transformer Architecture
-  Examples of LLMs: BERT and GPT
- How are LLMs Trained?
- Limitations and Challenges of LLMs

Encoder-only vs Decoder-only

BERT (Encoder-only)

- Sees the whole sentence at once.
- Good for understanding tasks.
- Classification, NER, search, Q&A.
- Does NOT generate text on its own.

GPT (Decoder-only)

- Reads left → right only (causal mask).
- Good for generation tasks.
- Chat, writing, code, completion.
- Powers ChatGPT, Claude, Gemini, LLaMA.

Topics

- What are Large Language Models?
- The Transformer Architecture
- Examples of LLMs: BERT and GPT
- How are LLMs Trained?
- Limitations and Challenges of LLMs

Pre-training

- Every LLM on the market has been **pre-trained** on a large corpus of text data and on specific language modeling-related tasks.
- During pre-training, the LLM tries to learn and understand general language and relationships between words.
- Every LLM is trained on different corpora and on different tasks.

Pre-training

- BERT was originally pre-trained on two publicly available text corpora:
  - English Wikipedia: A comprehensive collection from Wikipedia's English articles, spanning diverse topics and styles, encompassing about 2.5 billion words.
  - BookCorpus: An extensive dataset of both fiction and nonfiction books, covering various genres, totaling around 800 million words.

- Modern LLMs are trained on datasets vastly larger, thousands of times the size of traditional ones.

Pre-training

- BERT was pre-trained on two specific language modeling tasks:
  - Masked Language Modeling (MLM) task (autoencoding task): helps BERT recognize token interactions within a single sentence.
  - Next Sentence Prediction (NSP) task: helps BERT understand how tokens interact with each other between sentences.

Transfer Learning

- Transfer learning is a machine learning technique that harnesses knowledge from one task to enhance performance in a related task.
- For LLMs, it involves fine-tuning a pre-trained model for a specific task, like text classification or generation, utilizing its already acquired language knowledge.
- This approach significantly reduces training time and resources compared to training models from scratch.

Fine-Tuning

- Fine-tuning involves training the LLM on a smaller, task-specific dataset to adjust its parameters for the specific task at hand.
- This allows the LLM to leverage its pre-trained knowledge of the language to improve its accuracy for the specific task.
- Fine-tuning has been shown to drastically improve performance on domain-specific and task-specific tasks and lets LLMs adapt quickly to a wide variety of NLP applications.

Fine-Tuning Steps

1. Define the model we want to fine-tune as well as any fine-tuning parameters (e.g., learning rate).
2. Aggregate some training data (the format and other characteristics depend on the model we are updating).
3. Compute losses (a measure of error) and gradients (information about how to change the model to minimize error).
4. Update the model through backpropagation—a mechanism to update model parameters to minimize errors.

Topics

- What are Large Language Models?
- The Transformer Architecture
- Examples of LLMs: BERT and GPT
- How are LLMs Trained?
- Limitations and Challenges of LLMs

Limitations & Challenges of LLMs

- Hallucination — confidently make up facts.
- Knowledge cutoff — only knows what was in training data.
- Context window — limited memory per conversation (though growing fast).
- Cost & compute — training and serving are expensive and energy-hungry.
- Bias & safety — reflects biases in training data; can produce harmful output.
- Reasoning gaps — math, multi-step logic, planning are still imperfect.
- Reproducibility & evaluation — hard to benchmark “understanding”.
- Data & copyright — what was the model trained on, and is that OK?